

CISA

TOPNARY - T81TERNARY.CWEB



TERNARY_COMMON.H

Program Profiling Summary	3
File: ternary_system.cweb	3
General Stats	3
Key Components Identified	3
1. Axion Package Management & Optimization:	3
2. Ternary Arithmetic (T81BigInt & TritJS)	3
3. Memory and Optimization-Related Functions	3
4. I/O & Logging	4
5. Frequent Function Calls (≥ 3 times)	4

Program Profiling Summary

File: ternary_system.cweb

General Stats

- **Total Functions:** 78
- **Total Global Variables:** 205
- **Total Function Calls:** 628
- **Main Entry Points:**
 - **main()** (User-space entry point)
 - No explicit `init_module()` or `exit_module()`, suggesting it may not be a kernel module or these functions are indirectly referenced.

Key Components Identified

1. Axion Package Management & Optimization:

- `axion_install_pkg()`, `axion_update_pkg()`, `axion_rollback_pkg()`
- `axion_predict_needs()`, `axion_resolve_deps()`
- `axion_predictive_load_balancer()`
- `axion_get_perf_feedback()`, `axion_suggestion_work()`

2. Ternary Arithmetic (T81BigInt & TritJS)

- `tritjs_add_big()`, `tritjs_subtract_big()`, `tritjs_multiply_big()`
- `tritjs_divide_big()`, `tritjs_sqrt_complex()`, `tritjs_log3_complex()`
- `tritjs_factorial_big()`, `tritjs_power_big()`
- `binary_to_trit()`, `trit_to_binary()`

3. Memory and Optimization-Related Functions

- `allocate_digits()`, `karatsuba()` (Efficient multiplication)
- `multiply_with_cache()`, `mul_cache_lookup()`, `mul_cache_store()`
- **Frequent function calls:** `memset()`, `memcpy()`, `calloc()`, `free()`

4. I/O & Logging

- `printk()` (Kernel logging)
- `log_error()`
- `snprintf()`, `printw()`, `strncpy()`

5. Frequent Function Calls (≥ 3 times)

- **Common C Functions:** `memset()`, `memcpy()`, `calloc()`, `free()`, `printk()`, `strncpy()`, `snprintf()`
- **Performance-related:** `karatsuba()`, `allocate_digits()`, `multiply_with_cache()`
- **Axion functions:** `axion_install_pkg()`, `axion_predict_needs()`
- **Ternary Conversions:** `binary_to_trit()`, `trit_to_binary()`, `parse_trit_string()`

1. @* Ternary System: Unified Kernel Module and User-Space Utility
2.
3. This literate program integrates our Axion kernel module with the TritJS-CISA-Optimized
4.
5. user-space utility into one source file. Our mission is to transition from binary to
6.
7. native ternary computing by providing a unified foundation that spans both the kernel
8.
9. and user spaces.
10.
11.
12. FEATURES:
13.
14. - Common Ternary Logic Definitions:
15.
16. • Unified definitions for ternary states, instruction opcodes, and error codes.
17.
18. - Axion Kernel Module:
19.
20. • AI-powered predictive load balancing.
21.
22. • Ternary binary execution via JIT compilation (emulated on binary hardware).
23.
24. • Integrated package management with dependency resolution and rollback.
25.
26. - TritJS-CISA-Optimized Utility:
27.
28. • Advanced ternary arithmetic using optimized (Karatsuba) algorithms with caching.
29.
30. • Scientific functions (sqrt, log3, sin, cos, tan, π) via double conversion.
31.
32. • Secure audit logging and real-time intrusion detection.
33.
34. • An interactive, ncurses-based command-line interface.
35.
36. - Built-in Help System:
37.
38. • Type "help" at the command prompt to display a summary of available commands,
39.
40. usage examples, and detailed feature descriptions.
41.
42. - Unified Mission:
43.
44. • To provide a seamless, cohesive environment for testing and advancing ternary
45.
46. computing from the kernel all the way to the application level.
47.
48.
49. USAGE:
50.
51. • For kernel deployment: Compile with `__KERNEL__` defined so that the Axion module
52. is built.

53. • For user-space operation: Compile normally to build the TritJS-CISA-Optimized utility.

54.
55. • In the utility, type "help" at the prompt for a listing of commands and features.

56.
57.
58. This system is designed to evolve along with our research and development efforts,
59.
60. ensuring that both components work seamlessly toward our goal of native ternary computing.

61.
62. @*

63.

64.

65. /*

=====

66.
67. Section 1: Shared Header - ternary_common.h

68.
69. This header provides common definitions for ternary logic: ternary states,
70.
71. instruction opcodes for kernel execution, and error codes used by both parts.

72.

73.

=====

74.
75. @<ternary_common.h@>=

76.

77. /* ternary_common.h */

78.

79. #ifndef TERNARY_COMMON_H

80.

81. #define TERNARY_COMMON_H

82.

83.

84. /* Common definitions for ternary logic */

85.

86.

87. /* Ternary states */

88.

89. #define TERNARY_NEGATIVE -1 /* Represents -1: negative/false */

90.

91. #define TERNARY_ZERO 0 /* Represents 0: neutral/unknown */

92.

93. #define TERNARY_POSITIVE 1 /* Represents 1: positive/true */

94.

95.

96. /* Ternary instruction set opcodes for kernel execution */

97.

98. #define TADD 0x01 /* Add two trits (clamped between -1 and 1) */

99.

100. #define TSUB 0x02 /* Subtract two trits */

```

101.
102. #define TMUL 0x03 /* Multiply two trits */
103.
104. #define TAND 0x04 /* Ternary AND (minimum of two trits) */
105.
106. #define TOR 0x05 /* Ternary OR (maximum of two trits) */
107.
108. #define TNOT 0x06 /* Ternary NOT (negate a trit) */
109.
110. #define TJMP 0x07 /* Jump if condition is non-zero */
111.
112. #define TJZ 0x08 /* Jump if condition is zero */
113.
114. #define TJNZ 0x09 /* Jump if condition is non-zero */
115.
116. #define TLOAD 0x0A /* Load a trit from memory into a register */
117.
118. #define TSTORE 0x0B /* Store a trit from a register into memory */
119.
120. #define THLT 0x0C /* Halt execution */
121.
122.
123. /* Common error codes for ternary operations */
124.
125. typedef enum {
126.
127.     TERNARY_NO_ERROR = 0,
128.
129.     TERNARY_ERR_MEMALLOC,
130.
131.     TERNARY_ERR_INVALID_INPUT,
132.
133.     TERNARY_ERR_DIVZERO,
134.
135.     TERNARY_ERR_OVERFLOW,
136.
137.     TERNARY_ERR_UNDEFINED,
138.
139.     TERNARY_ERR_NEGATIVE,
140.
141.     TERNARY_ERR_PRECISION,
142.
143.     TERNARY_ERR_MMAP,
144.
145.     TERNARY_ERR_SCRIPT
146.
147. } TernaryError;
148.
149.
150. #endif /* TERNARY_COMMON_H */
151.
152.

```

```

153. @*
=====
=====
154.
155. Section 2: Kernel Module (Axion)
156.
157. This section implements the Axion kernel module. It provides:
158.
159. • AI-driven load balancing
160.
161. • Emulated ternary binary execution with a custom instruction set
162.
163. • Package management with dependency resolution and rollback
164.
165. This module is compiled only when __KERNEL__ is defined.
166.
167.
=====
===== */
168.
169. #ifdef __KERNEL__
170.
171. @<Axion Kernel Module@>=
172.
173. /*
174.
175. * Axion Kernel Module: AI-Powered Ternary Evolution
176.
177. *
178.
179. * Objective:
180.
181. * - Dynamically manage system resources with an AI-driven load balancer.
182.
183. * - Execute ternary binaries (emulated on binary hardware) using a dedicated
184.
185. * ternary instruction set.
186.
187. * - Provide package management with dependency resolution and rollback.
188.
189. *
190.
191. * This module uses common definitions from ternary_common.h and exposes IOCTL
192.
193. * interfaces and debugfs entries for telemetry and control.
194.
195. */
196.
197.
198. #include <linux/module.h>
199.
200. #include <linux/kernel.h>
201.
202. #include <linux/init.h>

```

```

203.
204. #include <linux/fs.h>
205.
206. #include <linux/cdev.h>
207.
208. #include <linux/uaccess.h>
209.
210. #include <linux/ioctl.h>
211.
212. #include <linux/device.h>
213.
214. #include <linux/kthread.h>
215.
216. #include <linux/delay.h>
217.
218. #include <linux/sched.h>
219.
220. #include <linux/debugfs.h>
221.
222. #include <linux/workqueue.h>
223.
224. #include <linux/cpu.h>
225.
226. #include <linux/notifier.h>
227.
228. #include <linux/topology.h>
229.
230. #include <linux/mm.h>
231.
232. #include <linux/slab.h>
233.
234. #include <linux/mempolicy.h>
235.
236. #include <linux/numa.h>
237.
238. #include <linux/random.h>
239.
240. #include <linux/binfmts.h>
241.
242. #include <linux/jiffies.h>
243.
244. #include <linux/timer.h>
245.
246. #include <asm/io.h>
247.
248. #include "ternary_common.h"
249.
250.
251. /* Module-specific constants and macros */
252.
253. #define DEVICE_NAME "axion_opt" /* Device name for character device */
254.
255. #define AXION_DEFAULT_REGISTER 0x1F /* Default register value */
256.

```

```

257. #define AXION_DEBUGFS_DIR "axion_debug"          /* Debugfs directory name */
258.
259. #define AXION_DEBUGFS_FILE "cpu_state"            /* Debugfs file for telemetry */
260.
261. #define WORKLOAD_HISTORY_SIZE 50                  /* History buffer size */
262.
263. #define ANOMALY_THRESHOLD 30                      /* Anomaly detection threshold (%)
*/
264.
265. #define SELF_HEALING_THRESHOLD 50                 /* Self-healing threshold (%) */
266.
267. #define PREDICTIVE_LOAD_BALANCING_INTERVAL 5000    /* Load balancing
interval (ms) */
268.
269. #define RESOURCE_WEIGHT_CPU 0.5                   /* Initial CPU weight */
270.
271. #define RESOURCE_WEIGHT_GPU 0.3                   /* Initial GPU weight */
272.
273. #define RESOURCE_WEIGHT_RAM 0.2                   /* Initial RAM weight */
274.
275. #define FEEDBACK_ADJUSTMENT_FACTOR 0.1            /* AI adjustment factor */
276.
277. #define MAX_PACKAGES 32                           /* Maximum packages tracked */
278.
279. #define MAX_DEPS 8                                /* Maximum dependencies per package */
280.
281. #define TBIN_MAGIC 0x5442494E                     /* Magic number for TBIN files */
282.
283. #define TERNARY_MEM_SIZE 32                       /* Size of ternary memory array */
284.
285.
286. /* Data structures */
287.
288. struct resource_state {
289.
290.     int cpu_usage;
291.
292.     int ram_usage;
293.
294.     int gpu_usage;
295.
296.     int action;
297.
298. };
299.
300.
301. struct rl_model {
302.
303.     int q_table[3][3];
304.
305.     int last_state;
306.
307.     int last_action;
308.

```

```

309.};
310.
311.
312.struct tbin_header {
313.
314.    uint32_t magic;
315.
316.    uint32_t entry_point;
317.
318.    uint32_t code_size;
319.
320.    uint32_t data_size;
321.
322.};
323.
324.
325.struct tbin_state {
326.
327.    int8_t reg[3];    /* Ternary registers */
328.
329.    int8_t memory[TERNARY_MEM_SIZE];
330.
331.    uint32_t ip;      /* Instruction pointer */
332.
333.    void *code;       /* Code buffer */
334.
335.    uint32_t code_size;
336.
337.    int running;      /* Running flag */
338.
339.};
340.
341.
342.struct package {
343.
344.    char name[32];
345.
346.    char version[16];
347.
348.    char deps[MAX_DEPS][32];
349.
350.    int dep_count;
351.
352.    int state;        /* Ternary state: -1, 0, or 1 */
353.
354.    int is_binary;
355.
356.    int risk_score;
357.
358.};
359.
360.
361.struct axion_state {
362.

```

```

363. struct resource_state res_history[WORKLOAD_HISTORY_SIZE];
364.
365. int res_history_index;
366.
367. struct rl_model rl;
368.
369. struct tbin_state tbin;
370.
371. int tbin_confidence_metric;
372.
373. int tbin_execution_profile;
374.
375. struct package packages[MAX_PACKAGES];
376.
377. int package_count;
378.
379. char suggestion[256];
380.
381. char last_cmd[256];
382.
383. unsigned long last_suggestion_time;
384.
385. int python_usage;
386.
387. int gaming_usage;
388.
389. uint64_t axion_register;
390.
391. int workload_history[WORKLOAD_HISTORY_SIZE];
392.
393. int workload_index;
394.
395. int adaptive_threshold;
396.
397. int confidence_metric;
398.
399. int rollback_counter;
400.
401. int rollback_reason[WORKLOAD_HISTORY_SIZE];
402.
403. bool rollback_suppression;
404.
405. int suppression_resistance;
406.
407. double resource_weight_cpu;
408.
409. double resource_weight_gpu;
410.
411. double resource_weight_ram;
412.
413. int resource_adjustment_log[WORKLOAD_HISTORY_SIZE];
414.
415. };
416.

```

```

417.
418. /* IOCTL command definitions */
419.
420. #define AXION_SET_REGISTER    _IOW('a', 1, uint64_t)
421.
422. #define AXION_GET_REGISTER    _IOR('a', 2, uint64_t)
423.
424. #define AXION_TBIN_LOAD      _IOW('a', 3, struct tbin_header)
425.
426. #define AXION_TBIN_STEP      _IO('a', 4)
427.
428. #define AXION_TBIN_GET_STATE  _IOR('a', 5, struct tbin_state)
429.
430. #define AXION_GET_SUGGESTION  _IOR('a', 6, char[256])
431.
432. #define AXION_INSTALL_PKG     _IOW('a', 7, char[32])
433.
434. #define AXION_UPDATE_PKG      _IOW('a', 8, char[32])
435.
436. #define AXION_SET_BINARY      _IOW('a', 9, int)
437.
438. #define AXION_ROLLBACK        _IOW('a', 10, char[32])
439.
440. #define AXION_NL_COMMAND      _IOW('a', 11, char[256])
441.
442. #define AXION_GET_PERF_FEEDBACK _IOR('a', 12, int)
443.
444.
445. /* Global variables for the module */
446.
447. static dev_t dev_num;
448.
449. static struct cdev axion_cdev;
450.
451. static struct class *axion_class;
452.
453. static struct device *axion_device;
454.
455. static struct task_struct *axion_thread;
456.
457. static struct dentry *debugfs_dir, *debugfs_file;
458.
459. static struct workqueue_struct *axion_wq;
460.
461. static struct work_struct axion_work;
462.
463. static struct timer_list axion_load_balancer;
464.
465. static struct axion_state state = {
466.
467.     .rl = { .q_table = {{5,2,1}, {3,5,2}, {1,3,5}}, .last_state = 0, .last_action = 0 },
468.
469.     .tbin_confidence_metric = 100,
470.

```

```

471. .tbin_execution_profile = 0,
472.
473. .axion_register = AXION_DEFAULT_REGISTER,
474.
475. .adaptive_threshold = SELF_HEALING_THRESHOLD,
476.
477. .confidence_metric = 100,
478.
479. .resource_weight_cpu = RESOURCE_WEIGHT_CPU,
480.
481. .resource_weight_gpu = RESOURCE_WEIGHT_GPU,
482.
483. .resource_weight_ram = RESOURCE_WEIGHT_RAM,
484.
485. .package_count = 0,
486.
487. .res_history_index = 0,
488.
489. .workload_index = 0,
490.
491. .rollback_counter = 0,
492.
493. .rollback_suppression = false,
494.
495. .suppression_resistance = 0
496.
497. };
498.
499.
500. /* Resource monitoring functions (simplified) */
501.
502. static int get_cpu_usage(void) { return 50; }
503.
504. static int get_ram_usage(void) { return 60; }
505.
506. static int get_gpu_usage(void) { return 40; }
507.
508.
509. static void axion_get_resource_usage(struct resource_state *res) {
510.
511.     res->cpu_usage = get_cpu_usage();
512.
513.     res->ram_usage = get_ram_usage();
514.
515.     res->gpu_usage = get_gpu_usage();
516.
517.     res->action = 0;
518.
519. }
520.
521.
522. static void axion_adjust_resource_weights(void) {
523.
524.     int cpu_usage = get_cpu_usage();

```

```

525.
526.     int gpu_usage = get_gpu_usage();
527.
528.     int ram_usage = get_ram_usage();
529.
530.     if (cpu_usage > gpu_usage && cpu_usage > ram_usage)
531.     {
532.         state.resource_weight_cpu += FEEDBACK_ADJUSTMENT_FACTOR;
533.     }
534.     else if (gpu_usage > cpu_usage && gpu_usage > ram_usage)
535.     {
536.         state.resource_weight_gpu += FEEDBACK_ADJUSTMENT_FACTOR;
537.     }
538.     else if (ram_usage > cpu_usage && ram_usage > gpu_usage)
539.     {
540.         state.resource_weight_ram += FEEDBACK_ADJUSTMENT_FACTOR;
541.     }
542.     double total = state.resource_weight_cpu + state.resource_weight_gpu +
state.resource_weight_ram;
543.
544.     state.resource_weight_cpu /= total;
545.
546.     state.resource_weight_gpu /= total;
547.
548.     state.resource_weight_ram /= total;
549.
550.     state.resource_adjustment_log[state.workload_index %
WORKLOAD_HISTORY_SIZE] =
551.     (int)(state.resource_weight_cpu * 100);
552.
553. }
554.
555.
556.
557. static void axion_predictive_load_balancer(struct timer_list *t) {
558.
559.     axion_adjust_resource_weights();
560.
561.     mod_timer(&axion_load_balancer, jiffies +
msecs_to_jiffies(PREDICTIVE_LOAD_BALANCING_INTERVAL));
562.
563. }
564.
565.
566. /* Ternary execution functions */
567.
568. static int axion_tbin_step(void) {
569.
570.     if (!state.tbin.running || state.tbin.ip >= state.tbin.code_size - 2 || !state.tbin.code) {
571.
572.         printk(KERN_ERR "Axion: Invalid TBIN state for execution\n");
573.
574.         return -EINVAL;
575.

```

```

576. }
577.
578. uint8_t *pc = (uint8_t *)state.tbin.code;
579.
580. uint8_t opcode = pc[state.tbin.ip];
581.
582. int8_t t1 = (pc[state.tbin.ip + 1] == 0xFF) ? TERNARY_NEGATIVE : pc[state.tbin.ip +
1];
583.
584. int8_t t2 = (pc[state.tbin.ip + 2] == 0xFF) ? TERNARY_NEGATIVE : pc[state.tbin.ip +
2];
585.
586. if (state.tbin.ip + 2 >= state.tbin.code_size) {
587.
588.     printk(KERN_ERR "Axion: TBIN IP out of bounds\n");
589.
590.     return -EFAULT;
591. }
592.
593.
594. switch (opcode) {
595.
596.     case TADD:
597.
598.         state.tbin.reg[0] += t1;
599.
600.         if (state.tbin.reg[0] > TERNARY_POSITIVE) state.tbin.reg[0] =
TERNARY_POSITIVE;
601.
602.         if (state.tbin.reg[0] < TERNARY_NEGATIVE) state.tbin.reg[0] =
TERNARY_NEGATIVE;
603.
604.         break;
605.
606.     case TSUB:
607.
608.         state.tbin.reg[0] -= t1;
609.
610.         if (state.tbin.reg[0] > TERNARY_POSITIVE) state.tbin.reg[0] =
TERNARY_POSITIVE;
611.
612.         if (state.tbin.reg[0] < TERNARY_NEGATIVE) state.tbin.reg[0] =
TERNARY_NEGATIVE;
613.
614.         break;
615.
616.     case TMUL:
617.
618.         state.tbin.reg[0] *= t1;
619.
620.         if (state.tbin.reg[0] > TERNARY_POSITIVE) state.tbin.reg[0] =
TERNARY_POSITIVE;
621.

```

```

622.         if (state.tbin.reg[0] < TERNARY_NEGATIVE) state.tbin.reg[0] =
TERNARY_NEGATIVE;
623.
624.         break;
625.
626.     case TAND:
627.
628.         state.tbin.reg[0] = (state.tbin.reg[0] < t1) ? state.tbin.reg[0] : t1;
629.
630.         break;
631.
632.     case TOR:
633.
634.         state.tbin.reg[0] = (state.tbin.reg[0] > t1) ? state.tbin.reg[0] : t1;
635.
636.         break;
637.
638.     case TNOT:
639.
640.         state.tbin.reg[0] = -t1;
641.
642.         break;
643.
644.     case TJMP:
645.
646.         if (t2 != 0 && t1 * 3 < state.tbin.code_size)
647.
648.             state.tbin.ip = t1 * 3;
649.
650.         else
651.
652.             return -EFAULT;
653.
654.         break;
655.
656.     case TJZ:
657.
658.         if (t2 == 0 && t1 * 3 < state.tbin.code_size)
659.
660.             state.tbin.ip = t1 * 3;
661.
662.         else
663.
664.             return -EFAULT;
665.
666.         break;
667.
668.     case TJNZ:
669.
670.         if (t2 != 0 && t1 * 3 < state.tbin.code_size)
671.
672.             state.tbin.ip = t1 * 3;
673.
674.         else

```

```

675.
676.     return -EFAULT;
677.
678.     break;
679.
680. case TLOAD:
681.
682.     if (t1 >= 0 && t1 < TERNARY_MEM_SIZE && t2 >= 0 && t2 < 3)
683.
684.         state.tbin.reg[t2] = state.tbin.memory[t1];
685.
686.     else
687.
688.         return -EFAULT;
689.
690.     break;
691.
692. case TSTORE:
693.
694.     if (t1 >= 0 && t1 < TERNARY_MEM_SIZE && t2 >= 0 && t2 < 3)
695.
696.         state.tbin.memory[t1] = state.tbin.reg[t2];
697.
698.     else
699.
700.         return -EFAULT;
701.
702.     break;
703.
704. case THLT:
705.
706.     state.tbin.running = 0;
707.
708.     printk(KERN_INFO "Axion: TBIN halted\n");
709.
710.     return 0;
711.
712. default:
713.
714.     printk(KERN_ERR "Axion: Unknown TBIN opcode 0x%x\n", opcode);
715.
716.     return -EINVAL;
717.
718. }
719.
720. state.tbin.ip += 3;
721.
722. return 0;
723.
724. }
725.
726.
727. static int axion_jit_compile_tbin(struct tbin_header *hdr) {
728.

```

```

729.  if (!hdr || hdr->code_size < 3) {
730.
731.      printk(KERN_ERR "Axion: Invalid TBIN header\n");
732.
733.      return -EINVAL;
734.
735.  }
736.
737.  if (state.tbin.code) {
738.
739.      vfree(state.tbin.code);
740.
741.      state.tbin.code = NULL;
742.
743.  }
744.
745.  int execution_efficiency = (hdr->code_size > 1024) ? TERNARY_POSITIVE :
TERNARY_NEGATIVE;
746.
747.  state.tbin_execution_profile = (state.tbin_execution_profile + execution_efficiency) /
2;
748.
749.  state.tbin_confidence_metric = (execution_efficiency !=
state.tbin_execution_profile) ?
750.
751.      state.tbin_confidence_metric - 5 : state.tbin_confidence_metric +
3;
752.
753.  if (state.tbin_confidence_metric > 100) state.tbin_confidence_metric = 100;
754.
755.  if (state.tbin_confidence_metric < 50) state.tbin_confidence_metric = 50;
756.
757.  state.tbin.code = vmalloc(hdr->code_size);
758.
759.  if (!state.tbin.code) {
760.
761.      printk(KERN_ERR "Axion: Failed to allocate TBIN memory\n");
762.
763.      return -ENOMEM;
764.
765.  }
766.
767.  if (copy_from_user(state.tbin.code, (void __user *)hdr->entry_point, hdr->code_size)) {
768.
769.      printk(KERN_ERR "Axion: Failed to copy TBIN code\n");
770.
771.      vfree(state.tbin.code);
772.
773.      state.tbin.code = NULL;
774.
775.      return -EFAULT;
776.
777.  }
778.

```

```

779.  if (hdr->code_size % 3 != 0) {
780.
781.      printk(KERN_ERR "Axion: Invalid TBIN code size\n");
782.
783.      vfree(state.tbin.code);
784.
785.      state.tbin.code = NULL;
786.
787.      return -EINVAL;
788.
789.  }
790.
791.  state.tbin.code_size = hdr->code_size;
792.
793.  state.tbin.ip = 0;
794.
795.  state.tbin.running = 1;
796.
797.  memset(state.tbin.reg, 0, sizeof(state.tbin.reg));
798.
799.  memset(state.tbin.memory, 0, sizeof(state.tbin.memory));
800.
801.  printk(KERN_INFO "Axion: TBIN loaded for ternary execution\n");
802.
803.  return 0;
804.
805.}
806.
807.
808. static int load_tbin_binary(struct linux_binprm *bprm) {
809.
810.     struct tbin_header hdr;
811.
812.     if (bprm->buf[0] != 'T' || bprm->buf[1] != 'B' ||
813.         bprm->buf[2] != 'I' || bprm->buf[3] != 'N')
814.         return -ENOEXEC;
815.
816.     return -ENOEXEC;
817.
818.     memcpy(&hdr, bprm->buf, sizeof(hdr));
819.
820.     if (hdr.magic != TBIN_MAGIC) return -ENOEXEC;
821.
822.     return axion_jit_compile_tbin(&hdr);
823.
824.}
825.
826.
827. static struct linux_binfmt axion_tbin_format = {
828.
829.     .module = THIS_MODULE,
830.
831.     .load_binary = load_tbin_binary,
832.

```

```

833.};
834.
835.
836./* Package management functions */
837.
838.static void axion_predict_needs(void) {
839.
840.    if (state.python_usage > 50)
841.
842.        snprintf(state.suggestion, sizeof(state.suggestion),
843.
844.            "Python usage high—install PyTorch or NumPy?");
845.
846.    else if (state.gaming_usage > 70)
847.
848.        snprintf(state.suggestion, sizeof(state.suggestion),
849.
850.            "Gaming detected—optimize with GPU tools?");
851.
852.    else
853.
854.        state.suggestion[0] = '\0';
855.
856.}
857.
858.
859.static int axion_resolve_deps(const char *pkg_name, int is_binary) {
860.
861.    if (!pkg_name) return -EINVAL;
862.
863.    for (int i = 0; i < state.package_count; i++) {
864.
865.        if (strcmp(state.packages[i].name, pkg_name) == 0 &&
866.
867.            state.packages[i].is_binary != is_binary) {
868.
869.            printk(KERN_WARNING "Axion: Dependency conflict for %s\n", pkg_name);
870.
871.            return -EAGAIN;
872.
873.        }
874.
875.    }
876.
877.    return 0;
878.
879.}
880.
881.
882.static int axion_install_pkg(const char *pkg_name, int is_binary) {
883.
884.    if (!pkg_name) return -EINVAL;
885.
886.    if (state.package_count >= MAX_PACKAGES) return -ENOMEM;

```

```

887.
888.     if (axion_resolve_deps(pkg_name, is_binary) < 0) return -EAGAIN;
889.
890.     struct package *pkg = &state.packages[state.package_count++];
891.
892.     strncpy(pkg->name, pkg_name, sizeof(pkg->name)-1);
893.
894.     pkg->name[sizeof(pkg->name)-1] = '\0';
895.
896.     snprintf(pkg->version, sizeof(pkg->version), "1.0.%d", (int)(get_random_int() % 10));
897.
898.     pkg->state = TERNARY_POSITIVE;
899.
900.     pkg->is_binary = is_binary;
901.
902.     pkg->risk_score = 0;
903.
904.     pkg->dep_count = 0;
905.
906.     if (strcmp(pkg_name, "python") == 0)
907.
908.         strncpy(pkg->deps[pkg->dep_count++], "libc", sizeof(pkg->deps[0]));
909.
910.     return 0;
911.
912. }
913.
914.
915. static int axion_update_pkg(const char *pkg_name) {
916.
917.     if (!pkg_name) return -EINVAL;
918.
919.     for (int i = 0; i < state.package_count; i++) {
920.
921.         if (strcmp(state.packages[i].name, pkg_name) == 0) {
922.
923.             int risk = (int)(get_random_int() % 100);
924.
925.             state.packages[i].risk_score = risk;
926.
927.             snprintf(state.packages[i].version, sizeof(state.packages[i].version),
928.
929.                 "1.1.%d", (int)(get_random_int() % 10));
930.
931.             if (risk > 70) {
932.
933.                 state.packages[i].state = TERNARY_ZERO;
934.
935.                 snprintf(state.suggestion, sizeof(state.suggestion),
936.
937.                     "Update to %s risky—rollback?", pkg_name);
938.
939.             } else {
940.

```

```

941.         state.packages[i].state = TERNARY_POSITIVE;
942.
943.     }
944.
945.     return 0;
946.
947. }
948.
949. }
950.
951. return -ENOENT;
952.
953.}
954.
955.
956. static int axion_rollback_pkg(const char *pkg_name) {
957.
958.     if (!pkg_name) return -EINVAL;
959.
960.     for (int i = 0; i < state.package_count; i++) {
961.
962.         if (strcmp(state.packages[i].name, pkg_name) == 0) {
963.
964.             snprintf(state.packages[i].version, sizeof(state.packages[i].version), "1.0.0");
965.
966.             state.packages[i].risk_score = 0;
967.
968.             state.packages[i].state = TERNARY_POSITIVE;
969.
970.             state.rollback_counter++;
971.
972.             state.rollback_reason[state.rollback_counter % WORKLOAD_HISTORY_SIZE] =
973. 1;
974.
975.             return 0;
976.         }
977.
978.     }
979.
980.     return -ENOENT;
981.
982.}
983.
984.
985. static int axion_nl_command(const char *cmd) {
986.
987.     if (!cmd) return -EINVAL;
988.
989.     strncpy(state.last_cmd, cmd, sizeof(state.last_cmd)-1);
990.
991.     state.last_cmd[sizeof(state.last_cmd)-1] = '\0';
992.
993.     if (strstr(cmd, "install a lightweight browser"))

```

```

994.
995.     return axion_install_pkg("lynx", 1);
996.
997. if (strstr(cmd, "update everything except my GPU drivers")) {
998.
999.     for (int i = 0; i < state.package_count; i++) {
1000.
1001.         if (strstr(state.packages[i].name, "nvidia") == NULL)
1002.
1003.             axion_update_pkg(state.packages[i].name);
1004.
1005.     }
1006.
1007.     return 0;
1008.
1009. }
1010.
1011. if (strstr(cmd, "optimize my gaming setup")) {
1012.
1013.     axion_install_pkg("nvidia-driver", 1);
1014.
1015.     snprintf(state.suggestion, sizeof(state.suggestion), "Gaming optimized!");
1016.
1017.     return 0;
1018.
1019. }
1020.
1021. return -EINVAL;
1022.
1023.}
1024.
1025.
1026. static int axion_get_perf_feedback(void) {
1027.
1028.     return (int)(get_random_int() % 100);
1029.
1030.}
1031.
1032.
1033. /* Workload and suggestion work function */
1034.
1035. static void axion_suggestion_work(struct work_struct *work) {
1036.
1037.     int current_load = state.workload_history[(state.workload_index == 0 ?
        WORKLOAD_HISTORY_SIZE - 1 : state.workload_index - 1)];
1038.
1039.     if (current_load > ANOMALY_THRESHOLD)
1040.
1041.         printk(KERN_WARNING "Axion: Anomaly detected - Load: %d%%\n",
            current_load);
1042.
1043.     axion_predict_needs();
1044.
1045.     schedule_work(&axion_work);

```

```

1046.
1047.}
1048.
1049.
1050./* Module initialization and exit functions */
1051.
1052.static int __init axion_init(void) {
1053.
1054.    int ret = alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
1055.
1056.    if (ret < 0) {
1057.
1058.        printk(KERN_ERR "Axion: Failed to allocate device number\n");
1059.
1060.        return ret;
1061.
1062.    }
1063.
1064.    cdev_init(&axion_cdev, NULL);
1065.
1066.    ret = cdev_add(&axion_cdev, dev_num, 1);
1067.
1068.    if (ret < 0) {
1069.
1070.        printk(KERN_ERR "Axion: Failed to add cdev\n");
1071.
1072.        unregister_chrdev_region(dev_num, 1);
1073.
1074.        return ret;
1075.
1076.    }
1077.
1078.    axion_class = class_create(THIS_MODULE, DEVICE_NAME);
1079.
1080.    if (IS_ERR(axion_class)) {
1081.
1082.        cdev_del(&axion_cdev);
1083.
1084.        unregister_chrdev_region(dev_num, 1);
1085.
1086.        return PTR_ERR(axion_class);
1087.
1088.    }
1089.
1090.    axion_device = device_create(axion_class, NULL, dev_num, NULL,
        DEVICE_NAME);
1091.
1092.    if (IS_ERR(axion_device)) {
1093.
1094.        class_destroy(axion_class);
1095.
1096.        cdev_del(&axion_cdev);
1097.
1098.        unregister_chrdev_region(dev_num, 1);

```

```

1099.
1100.     return PTR_ERR(axion_device);
1101.
1102. }
1103.
1104.     debugfs_dir = debugfs_create_dir(AXION_DEBUGFS_DIR, NULL);
1105.
1106.     debugfs_file = debugfs_create_file(AXION_DEBUGFS_FILE, 0444, debugfs_dir,
        &state, NULL);
1107.
1108.     timer_setup(&axion_load_balancer, axion_predictive_load_balancer, 0);
1109.
1110.     mod_timer(&axion_load_balancer, jiffies +
        msecs_to_jiffies(PREDICTIVE_LOAD_BALANCING_INTERVAL));
1111.
1112.     axion_wq = create_singlethread_workqueue("axion_wq");
1113.
1114.     INIT_WORK(&axion_work, axion_suggestion_work);
1115.
1116.     printk(KERN_INFO "Axion: Module initialized\n");
1117.
1118.     return 0;
1119.
1120.}
1121.
1122.
1123. static void __exit axion_exit(void) {
1124.
1125.     del_timer_sync(&axion_load_balancer);
1126.
1127.     device_destroy(axion_class, dev_num);
1128.
1129.     class_destroy(axion_class);
1130.
1131.     cdev_del(&axion_cdev);
1132.
1133.     unregister_chrdev_region(dev_num, 1);
1134.
1135.     debugfs_remove_recursive(debugfs_dir);
1136.
1137.     flush_workqueue(axion_wq);
1138.
1139.     destroy_workqueue(axion_wq);
1140.
1141.     printk(KERN_INFO "Axion: Module exited\n");
1142.
1143.}
1144.
1145.
1146. module_init(axion_init);
1147.
1148. module_exit(axion_exit);
1149.
1150. MODULE_LICENSE("GPL");

```

```

1151.
1152. MODULE_AUTHOR("Your Name");
1153.
1154. MODULE_DESCRIPTION("Axion: AI-Powered Kernel Module for Ternary Logic
    Evolution");
1155.
1156.
1157. /* End of Kernel Module section. */
1158.
1159. #endif /* __KERNEL__ */
1160.
1161.
1162. /*
    =====
    =====
1163.
1164. Section 3: User-Space Utility Program (TritJS-CISA-Optimized)
1165.
1166. This section implements our advanced ternary calculator utility with an
1167.
1168. interactive ncurses-based interface. It supports advanced arithmetic,
1169.
1170. scientific functions, secure logging, and a built-in help system (type "help").
1171.
1172. This section is compiled when __KERNEL__ is not defined.
1173.
1174.
    =====
    ===== */
1175.
1176. #ifndef __KERNEL__
1177.
1178. @<TritJS-CISA-Optimized Utility@>=
1179.
1180. /*****
1181.
1182. * TritJS-CISA-Optimized: Advanced Ternary Calculator Utility
1183.
1184. *
1185.
1186. * FEATURES:
1187.
1188. * - Safe dynamic memory management and optimized arithmetic operations,
1189.
1190. * including Karatsuba multiplication with caching.
1191.
1192. * - Scientific functions (sqrt, log3, sin, cos, tan,  $\pi$ ) via double conversion.
1193.
1194. * - Secure audit logging and simulated intrusion detection.
1195.
1196. * - Interactive ncurses-based command-line interface with dynamic resizing.
1197.
1198. * - Built-in help system: Type "help" at the prompt for command details.
1199.

```

```

1200. *
1201.
1202. * USAGE:
1203.
1204. * Compile with:
1205.
1206. * gcc -DUSE_READLINE -o tritjs_cisa_optimized ternary_system.cweb -lm
    -lreadline \
1207.
1208. *      -fstack-protector-strong -D_FORTIFY_SOURCE=2 -pie -fPIE -lncurses -lcrypto
    -lpthread -llua
1209.
1210. * Then run:
1211.
1212. *      ./tritjs_cisa_optimized
1213.
1214. *
1215.
1216. * This utility shares a unified ternary logic foundation with the Axion kernel module,
1217.
1218. * helping to advance our mission toward native ternary computing.
1219.
1220. *****/
1221.
1222.
1223. #include <stdio.h>
1224.
1225. #include <stdlib.h>
1226.
1227. #include <string.h>
1228.
1229. #include <math.h>
1230.
1231. #include <sys/mman.h>
1232.
1233. #include <fcntl.h>
1234.
1235. #include <unistd.h>
1236.
1237. #include <time.h>
1238.
1239. #include <limits.h>
1240.
1241. #include <ncurses.h>
1242.
1243. #include <errno.h>
1244.
1245. #include <sys/file.h>
1246.
1247. #include <openssl/evp.h>
1248.
1249. #include <pthread.h>
1250.
1251. #include <lua.h>

```

```

1252.
1253. #include <lualib.h>
1254.
1255. #include <lauxlib.h>
1256.
1257. #include "ternary_common.h"
1258.
1259.
1260. #ifdef USE_READLINE
1261.
1262. #include <readline/readline.h>
1263.
1264. #include <readline/history.h>
1265.
1266. #endif
1267.
1268.
1269. /* Global configuration */
1270.
1271. #define ENABLE_VERBOSE_LOGGING 1
1272.
1273. #define VERSION "2.0-upgrade-optimized"
1274.
1275. #define BASE_81 81
1276.
1277. #define T81_MMAP_THRESHOLD (500 * 1024)
1278.
1279.
1280. /* Data structures for big integer arithmetic */
1281.
1282. typedef struct {
1283.
1284.     int sign;          /* 0 = positive, 1 = negative */
1285.
1286.     unsigned char *digits; /* Array of base-81 digits (little-endian) */
1287.
1288.     size_t len;        /* Number of digits */
1289.
1290.     int is_mapped;      /* 1 if allocated with mmap */
1291.
1292.     int fd;             /* File descriptor if using mmap */
1293.
1294.     char tmp_path[32];  /* Temporary file path */
1295.
1296. } T81BigInt;
1297.
1298.
1299. typedef struct {
1300.
1301.     int sign;
1302.
1303.     unsigned char* integer; /* Digits for integer part */
1304.
1305.     unsigned char* fraction; /* Digits for fractional part */

```

```

1306.
1307.     size_t i_len, f_len;
1308.
1309.     int i_mapped, f_mapped;
1310.
1311.     int i_fd, f_fd;
1312.
1313.     char i_tmp_path[32];
1314.
1315.     char f_tmp_path[32];
1316.
1317. } T81Float;
1318.
1319.
1320. typedef struct {
1321.
1322.     T81Float real;
1323.
1324.     T81Float imag;
1325.
1326. } T81Complex;
1327.
1328.
1329. typedef struct {
1330.
1331.     T81Float quotient;
1332.
1333.     T81Float remainder;
1334.
1335. } T81DivResult;
1336.
1337.
1338. #define MAX_SCRIPT_NAME 10
1339.
1340. #define MAX_SCRIPT_CMDS 50
1341.
1342. typedef struct {
1343.
1344.     char name[MAX_SCRIPT_NAME];
1345.
1346.     char commands[MAX_SCRIPT_CMDS][256];
1347.
1348.     int cmd_count;
1349.
1350. } Script;
1351.
1352.
1353. /* Global variables for the utility */
1354.
1355. static FILE* audit_log = NULL;
1356.
1357. static long total_mapped_bytes = 0;
1358.
1359. static int operation_steps = 0;

```

```

1360.
1361. #define MAX_HISTORY 10
1362.
1363. static char* history[MAX_HISTORY] = {0};
1364.
1365. static int history_count = 0;
1366.
1367. static T81BigInt* variables[26] = {0};
1368.
1369. static Script scripts[10] = {0};
1370.
1371. static int script_count = 0;
1372.
1373. static WINDOW *input_win, *output_win, *status_win;
1374.
1375.
1376. /* Function prototypes for arithmetic and scientific operations */
1377.
1378. TernaryError tritjs_add_big(T81BigInt* A, T81BigInt* B, T81BigInt** result);
1379.
1380. TernaryError tritjs_subtract_big(T81BigInt* A, T81BigInt* B, T81BigInt** result);
1381.
1382. TernaryError tritjs_multiply_big(T81BigInt* a, T81BigInt* b, T81BigInt** result);
1383.
1384. TernaryError tritjs_factorial_big(T81BigInt* a, T81BigInt** result);
1385.
1386. TernaryError tritjs_power_big(T81BigInt* base, T81BigInt* exp, T81BigInt** result);
1387.
1388. TernaryError tritjs_divide_big(T81BigInt* a, T81BigInt* b, T81BigInt** quotient,
    T81BigInt** remainder);
1389.
1390. TernaryError tritjs_sqrt_complex(T81BigInt* a, int precision, T81Complex* result);
1391.
1392. TernaryError tritjs_log3_complex(T81BigInt* a, int precision, T81Complex* result);
1393.
1394. TernaryError tritjs_sin_complex(T81BigInt* a, int precision, T81Complex* result);
1395.
1396. TernaryError tritjs_cos_complex(T81BigInt* a, int precision, T81Complex* result);
1397.
1398. TernaryError tritjs_tan_complex(T81BigInt* a, int precision, T81Complex* result);
1399.
1400. TernaryError tritjs_pi(int* len, int** pi);
1401.
1402. TernaryError parse_trit_string(const char* s, T81BigInt** out);
1403.
1404. TernaryError t81bigint_to_trit_string(const T81BigInt* in, char** out);
1405.
1406. TernaryError binary_to_trit(int num, T81BigInt** out);
1407.
1408. TernaryError trit_to_binary(T81BigInt* x, int* outVal);
1409.
1410. void tritbig_free(T81BigInt* x);
1411.
1412. TernaryError parse_balanced_trit_string(const char* s, T81BigInt** out);

```

```

1413.
1414.
1415. /* Logging and error handling functions */
1416.
1417. static const char* trit_error_str(TernaryError err) {
1418.
1419.     switch(err){
1420.
1421.         case TERNARY_NO_ERROR: return "No error";
1422.
1423.         case TERNARY_ERR_MEMALLOC: return "Memory allocation failed";
1424.
1425.         case TERNARY_ERR_INVALID_INPUT: return "Invalid input";
1426.
1427.         case TERNARY_ERR_DIVZERO: return "Division by zero";
1428.
1429.         case TERNARY_ERR_OVERFLOW: return "Overflow detected";
1430.
1431.         case TERNARY_ERR_UNDEFINED: return "Operation undefined";
1432.
1433.         case TERNARY_ERR_NEGATIVE: return "Negative input";
1434.
1435.         case TERNARY_ERR_PRECISION: return "Precision limit exceeded";
1436.
1437.         case TERNARY_ERR_MMAP: return "Memory mapping failed";
1438.
1439.         case TERNARY_ERR_SCRIPT: return "Script error";
1440.
1441.         default: return "Unknown error";
1442.
1443.     }
1444.
1445. }
1446.
1447.
1448. static void log_error(TernaryError err, const char* context, const char* file, int line) {
1449.
1450.     if (!audit_log) return;
1451.
1452.     time_t now;
1453.
1454.     time(&now);
1455.
1456.     fprintf(audit_log, "[%s] ERROR %d: %s in %s (%s:%d)\n",
1457.
1458.         ctime(&now), err, trit_error_str(err), context, file, line);
1459.
1460.     fflush(audit_log);
1461.
1462. }
1463.
1464.
1465. /* Memory management for T81BigInt */
1466.

```

```

1467. static TernaryError allocate_digits(T81BigInt *x, size_t lengthNeeded) {
1468.
1469.     size_t bytesNeeded = (lengthNeeded == 0 ? 1 : lengthNeeded);
1470.
1471.     x->len = lengthNeeded;
1472.
1473.     x->is_mapped = 0;
1474.
1475.     x->fd = -1;
1476.
1477.     if (bytesNeeded < T81_MMAP_THRESHOLD) {
1478.
1479.         x->digits = (unsigned char*)calloc(bytesNeeded, 1);
1480.
1481.         if (!x->digits) return TERNARY_ERR_MEMALLOC;
1482.
1483.         return TERNARY_NO_ERROR;
1484.
1485.     }
1486.
1487.     strcpy(x->tmp_path, "/tmp/tritjs_cisa_XXXXXX");
1488.
1489.     x->fd = mkstemp(x->tmp_path);
1490.
1491.     if (x->fd < 0) return TERNARY_ERR_MMAP;
1492.
1493.     if (ftruncate(x->fd, bytesNeeded) < 0) {
1494.
1495.         close(x->fd);
1496.
1497.         return TERNARY_ERR_MMAP;
1498.
1499.     }
1500.
1501.     x->digits = mmap(NULL, bytesNeeded, PROT_READ | PROT_WRITE,
MAP_SHARED, x->fd, 0);
1502.
1503.     if (x->digits == MAP_FAILED) {
1504.
1505.         close(x->fd);
1506.
1507.         return TERNARY_ERR_MMAP;
1508.
1509.     }
1510.
1511.     unlink(x->tmp_path);
1512.
1513.     x->is_mapped = 1;
1514.
1515.     total_mapped_bytes += bytesNeeded;
1516.
1517.     operation_steps++;
1518.
1519.     return TERNARY_NO_ERROR;

```

```

1520.
1521.}
1522.
1523.
1524.static void t81bigint_free(T81BigInt* x) {
1525.
1526.    if (!x) return;
1527.
1528.    if (x->is_mapped && x->digits && x->digits != MAP_FAILED) {
1529.
1530.        size_t bytes = (x->len == 0 ? 1 : x->len);
1531.
1532.        munmap(x->digits, bytes);
1533.
1534.        close(x->fd);
1535.
1536.        total_mapped_bytes -= bytes;
1537.
1538.        operation_steps++;
1539.
1540.    } else {
1541.
1542.        free(x->digits);
1543.
1544.    }
1545.
1546.}
1547.
1548.
1549./* Audit logging initialization */
1550.
1551.static void init_audit_log() {
1552.
1553.    audit_log = fopen("/var/log/tritjs_cisa.log", "a");
1554.
1555.    if (!audit_log) {
1556.
1557.        perror("Audit log init failed; fallback to stderr");
1558.
1559.        audit_log = stderr;
1560.
1561.    } else {
1562.
1563.        flock(fileno(audit_log), LOCK_EX);
1564.
1565.    }
1566.
1567.}
1568.
1569.
1570./* Base conversion and parsing functions */
1571.
1572.static TernaryError parse_trit_string_base81_optimized(const char* str, T81BigInt* out)
    {

```

```

1573.
1574.  if (!str || !str[0]) return TERNARY_ERR_INVALID_INPUT;
1575.
1576.  memset(out, 0, sizeof(*out));
1577.
1578.  int sign = 0;
1579.
1580.  size_t pos = 0;
1581.
1582.  if (str[0] == '-' || str[0] == '+') { sign = 1; pos = 1; }
1583.
1584.  size_t total_len = strlen(str) - pos;
1585.
1586.  size_t remainder = total_len % 4;
1587.
1588.  if (allocate_digits(out, 1) != TERNARY_NO_ERROR) return
    TERNARY_ERR_MEMALLOC;
1589.
1590.  out->digits[0] = 0;
1591.
1592.  out->sign = sign;
1593.
1594.  for (size_t i = 0; i < remainder; i++) {
1595.
1596.      int digit = str[pos + i] - '0';
1597.
1598.      if (digit < 0 || digit > 2) return TERNARY_ERR_INVALID_INPUT;
1599.
1600.      int carry = digit;
1601.
1602.      for (size_t j = 0; j < out->len; j++) {
1603.
1604.          int val = out->digits[j] * 3 + carry;
1605.
1606.          out->digits[j] = val % BASE_81;
1607.
1608.          carry = val / BASE_81;
1609.
1610.      }
1611.
1612.      while (carry) {
1613.
1614.          size_t old_len = out->len;
1615.
1616.          if (allocate_digits(out, out->len + 1) != TERNARY_NO_ERROR) return
    TERNARY_ERR_MEMALLOC;
1617.
1618.          out->digits[old_len] = carry % BASE_81;
1619.
1620.          carry /= BASE_81;
1621.
1622.      }
1623.
1624.  }

```

```

1625.
1626.     pos += remainder;
1627.
1628.     while (pos < strlen(str)) {
1629.
1630.         int groupVal = 0;
1631.
1632.         for (int k = 0; k < 4; k++) {
1633.
1634.             if (str[pos + k] < '0' || str[pos + k] > '2') return TERNARY_ERR_INVALID_INPUT;
1635.
1636.             groupVal = groupVal * 3 + (str[pos + k] - '0');
1637.
1638.         }
1639.
1640.         pos += 4;
1641.
1642.         int carry = groupVal;
1643.
1644.         for (size_t j = 0; j < out->len; j++) {
1645.
1646.             int val = out->digits[j] * 81 + carry;
1647.
1648.             out->digits[j] = val % BASE_81;
1649.
1650.             carry = val / BASE_81;
1651.
1652.         }
1653.
1654.         while (carry) {
1655.
1656.             size_t old_len = out->len;
1657.
1658.             if (allocate_digits(out, out->len + 1) != TERNARY_NO_ERROR) return
TERNARY_ERR_MEMALLOC;
1659.
1660.             out->digits[old_len] = carry % BASE_81;
1661.
1662.             carry /= BASE_81;
1663.
1664.         }
1665.
1666.     }
1667.
1668.     while (out->len > 1 && out->digits[out->len - 1] == 0)
1669.
1670.         out->len--;
1671.
1672.     return TERNARY_NO_ERROR;
1673.
1674. }
1675.
1676.
1677. static TernaryError parse_trit_string(const char* s, T81BigInt** out) {

```

```

1678.
1679.     if (!out) return TERNARY_ERR_INVALID_INPUT;
1680.
1681.     *out = (T81BigInt*)calloc(1, sizeof(T81BigInt));
1682.
1683.     if (!*out) return TERNARY_ERR_MEMALLOC;
1684.
1685.     TernaryError e = parse_trit_string_base81_optimized(s, *out);
1686.
1687.     if (e != TERNARY_NO_ERROR) { free(*out); *out = NULL; }
1688.
1689.     return e;
1690.
1691. }
1692.
1693.
1694. static TernaryError t81bigint_to_trit_string(const T81BigInt* in, char** out) {
1695.
1696.     if (!in || !out) return TERNARY_ERR_INVALID_INPUT;
1697.
1698.     if (in->len == 1 && in->digits[0] == 0) {
1699.
1700.         *out = strdup("0");
1701.
1702.         return TERNARY_NO_ERROR;
1703.     }
1704.
1705.
1706.     T81BigInt tmp = *in;
1707.
1708.     T81BigInt tmpCopy;
1709.
1710.     memset(&tmpCopy, 0, sizeof(tmpCopy));
1711.
1712.     if (allocate_digits(&tmpCopy, tmp.len) != TERNARY_NO_ERROR) return
        TERNARY_ERR_MEMALLOC;
1713.
1714.     tmpCopy.len = tmp.len;
1715.
1716.     memcpy(tmpCopy.digits, tmp.digits, tmp.len);
1717.
1718.     tmpCopy.sign = tmp.sign;
1719.
1720.     size_t capacity = tmp.len * 4 + 2;
1721.
1722.     char* buf = calloc(capacity, 1);
1723.
1724.     if (!buf) { t81bigint_free(&tmpCopy); return TERNARY_ERR_MEMALLOC; }
1725.
1726.     size_t idx = 0;
1727.
1728.     while (1) {
1729.
1730.         int isZero = 1;

```

```

1731.
1732.     for (size_t i = 0; i < tmpCopy.len; i++) {
1733.
1734.         if (tmpCopy.digits[i] != 0) { isZero = 0; break; }
1735.
1736.     }
1737.
1738.     if (isZero) { if (idx == 0) buf[idx++] = '0'; break; }
1739.
1740.     int carry = 0;
1741.
1742.     for (ssize_t i = tmpCopy.len - 1; i >= 0; i--) {
1743.
1744.         int val = tmpCopy.digits[i] + carry * BASE_81;
1745.
1746.         int q = val / 3;
1747.
1748.         int r = val % 3;
1749.
1750.         tmpCopy.digits[i] = q;
1751.
1752.         carry = r;
1753.
1754.     }
1755.
1756.     buf[idx++] = (char)('0' + carry);
1757.
1758. }
1759.
1760. t81bigint_free(&tmpCopy);
1761.
1762. if (in->sign) { buf[idx++] = '-'; }
1763.
1764. for (size_t i = 0; i < idx / 2; i++) {
1765.
1766.     char t = buf[i];
1767.
1768.     buf[i] = buf[idx - 1 - i];
1769.
1770.     buf[idx - 1 - i] = t;
1771.
1772. }
1773.
1774. buf[idx] = '\0';
1775.
1776. *out = buf;
1777.
1778. return TERNARY_NO_ERROR;
1779.
1780.}
1781.
1782.
1783. static TernaryError binary_to_trit(int num, T81BigInt** out) {
1784.

```

```

1785. char b3[128];
1786.
1787. int sign = (num < 0) ? 1 : 0;
1788.
1789. int val = (num < 0) ? -num : num;
1790.
1791. size_t idx = 0;
1792.
1793. if (val == 0) { b3[idx++] = '0'; }
1794.
1795. while (val > 0) { int r = val % 3; b3[idx++] = (char)('0' + r); val /= 3; }
1796.
1797. if (idx == 0) { b3[idx++] = '0'; }
1798.
1799. if (sign) { b3[idx++] = '-'; }
1800.
1801. for (size_t i = 0; i < idx / 2; i++) {
1802.
1803.     char t = b3[i];
1804.
1805.     b3[i] = b3[idx - 1 - i];
1806.
1807.     b3[idx - 1 - i] = t;
1808.
1809. }
1810.
1811. b3[idx] = '\0';
1812.
1813. return parse_trit_string(b3, out);
1814.
1815.}
1816.
1817.
1818. static TernaryError trit_to_binary(T81BigInt* x, int* outVal) {
1819.
1820.     char* b3 = NULL;
1821.
1822.     if (t81bigint_to_trit_string(x, &b3) != TERNARY_NO_ERROR) return
        TERNARY_ERR_INVALID_INPUT;
1823.
1824.     long long accum = 0;
1825.
1826.     int sign = 0;
1827.
1828.     size_t i = 0;
1829.
1830.     if (b3[0] == '-') { sign = 1; i = 1; }
1831.
1832.     for (; b3[i]; i++) {
1833.
1834.         if (b3[i] < '0' || b3[i] > '2') { free(b3); return TERNARY_ERR_INVALID_INPUT; }
1835.
1836.         accum = accum * 3 + (b3[i] - '0');
1837.

```

```

1838.     if (accum > INT_MAX) { free(b3); return TERNARY_ERR_OVERFLOW; }
1839.
1840. }
1841.
1842. free(b3);
1843.
1844. if (sign) accum = -accum;
1845.
1846. *outVal = (int)accum;
1847.
1848. return TERNARY_NO_ERROR;
1849.
1850.}
1851.
1852.
1853. void tritbig_free(T81BigInt* x) {
1854.
1855.     if (!x) return;
1856.
1857.     t81bigint_free(x);
1858.
1859.     free(x);
1860.
1861.}
1862.
1863.
1864. /* Balanced ternary parsing: convert balanced ternary string to T81BigInt */
1865.
1866. TernaryError parse_balanced_trit_string(const char* s, T81BigInt** out) {
1867.
1868.     if (!s) return TERNARY_ERR_INVALID_INPUT;
1869.
1870.     size_t len = strlen(s);
1871.
1872.     char* unb = calloc(len + 1, 1);
1873.
1874.     if (!unb) return TERNARY_ERR_MEMALLOC;
1875.
1876.     for (size_t i = 0; i < len; i++) {
1877.
1878.         char c = s[i];
1879.
1880.         if (c == '-' || c == '-') { unb[i] = '0'; }
1881.
1882.         else if (c == '0') { unb[i] = '1'; }
1883.
1884.         else if (c == '+') { unb[i] = '2'; }
1885.
1886.         else { free(unb); return TERNARY_ERR_INVALID_INPUT; }
1887.
1888.     }
1889.
1890.     unb[len] = '\0';
1891.

```

```

1892. TernaryError e = parse_trit_string(unb, out);
1893.
1894. free(unb);
1895.
1896. return e;
1897.
1898.}
1899.
1900.
1901./* Arithmetic operations implementation */
1902.
1903.
1904./* Addition */
1905.
1906.TernaryError tritjs_add_big(T81BigInt* A, T81BigInt* B, T81BigInt** result) {
1907.
1908.    if (!A || !B) return TERNARY_ERR_INVALID_INPUT;
1909.
1910.    *result = (T81BigInt*)calloc(1, sizeof(T81BigInt));
1911.
1912.    if (!*result) return TERNARY_ERR_MEMALLOC;
1913.
1914.    if (A->sign == B->sign) {
1915.
1916.        (*result)->sign = A->sign;
1917.
1918.        size_t len = (A->len > B->len ? A->len : B->len) + 1;
1919.
1920.        if (allocate_digits(*result, len) != TERNARY_NO_ERROR) { free(*result); return
TERNARY_ERR_MEMALLOC; }
1921.
1922.        memset((*result)->digits, 0, len);
1923.
1924.        memcpy((*result)->digits, A->digits, A->len);
1925.
1926.        for (size_t i = 0; i < B->len; i++) {
1927.
1928.            int val = (*result)->digits[i] + B->digits[i];
1929.
1930.            (*result)->digits[i] = val % BASE_81;
1931.
1932.            int carry = val / BASE_81;
1933.
1934.            size_t cpos = i + 1;
1935.
1936.            while (carry && cpos < len) {
1937.
1938.                val = (*result)->digits[cpos] + carry;
1939.
1940.                (*result)->digits[cpos] = val % BASE_81;
1941.
1942.                carry = val / BASE_81;
1943.
1944.                cpos++;

```

```

1945.
1946.     }
1947.
1948.     }
1949.
1950.     while ((*result)->len > 1 && (*result)->digits[(*result)->len - 1] == 0)
1951.
1952.         (*result)->len--;
1953.
1954.     } else {
1955.
1956.         int c = memcmp(A->digits, B->digits, (A->len < B->len ? A->len : B->len));
1957.
1958.         T81BigInt *larger, *smaller;
1959.
1960.         int largerSign;
1961.
1962.         if (c > 0) { larger = A; smaller = B; largerSign = A->sign; }
1963.
1964.         else if (c < 0) { larger = B; smaller = A; largerSign = B->sign; }
1965.
1966.         else {
1967.
1968.             if (allocate_digits(*result, 1) != TERNARY_NO_ERROR) { free(*result); return
TERNARY_ERR_MEMALLOC; }
1969.
1970.             (*result)->digits[0] = 0;
1971.
1972.             return TERNARY_NO_ERROR;
1973.
1974.         }
1975.
1976.         (*result)->sign = largerSign;
1977.
1978.         if (allocate_digits(*result, larger->len) != TERNARY_NO_ERROR) { free(*result);
return TERNARY_ERR_MEMALLOC; }
1979.
1980.         memcpy((*result)->digits, larger->digits, larger->len);
1981.
1982.         for (size_t i = 0; i < smaller->len; i++) {
1983.
1984.             int diff = (*result)->digits[i] - smaller->digits[i];
1985.
1986.             if (diff < 0) {
1987.
1988.                 diff += BASE_81;
1989.
1990.                 size_t j = i + 1;
1991.
1992.                 while (1) {
1993.
1994.                     (*result)->digits[j]--;
1995.
1996.                     if ((*result)->digits[j] < 255) break;

```

```

1997.
1998.         (*result)->digits[j] += BASE_81;
1999.
2000.         j++;
2001.
2002.         if (j >= larger->len) break;
2003.
2004.     }
2005.
2006. }
2007.
2008.     (*result)->digits[i] = (unsigned char)diff;
2009.
2010. }
2011.
2012. while ((*result)->len > 1 && (*result)->digits[(*result)->len - 1] == 0)
2013.
2014.     (*result)->len--;
2015.
2016. }
2017.
2018. return TERNARY_NO_ERROR;
2019.
2020.}
2021.
2022.
2023./* Subtraction */
2024.
2025.TernaryError tritjs_subtract_big(T81BigInt* A, T81BigInt* B, T81BigInt** result) {
2026.
2027.    if (!A || !B) return TERNARY_ERR_INVALID_INPUT;
2028.
2029.    T81BigInt tmp = *B;
2030.
2031.    int oldsign = tmp.sign;
2032.
2033.    tmp.sign = !oldsign;
2034.
2035.    TernaryError e = tritjs_add_big(A, &tmp, result);
2036.
2037.    tmp.sign = oldsign;
2038.
2039.    return e;
2040.
2041.}
2042.
2043.
2044./* Multiplication using Karatsuba algorithm with caching */
2045.
2046.#define MUL_CACHE_SIZE 8
2047.
2048.typedef struct {
2049.
2050.    char key[128];

```

```

2051.
2052.     T81BigInt result;
2053.
2054.     int used;
2055.
2056. } MulCacheEntry;
2057.
2058. static MulCacheEntry mul_cache[MUL_CACHE_SIZE] = {{0}};
2059.
2060.
2061. static void naive_mul(const unsigned char *A, size_t alen,
2062.
2063.                     const unsigned char *B, size_t blen,
2064.
2065.                     unsigned char *out) {
2066.
2067.     memset(out, 0, alen + blen);
2068.
2069.     for (size_t i = 0; i < alen; i++) {
2070.
2071.         int carry = 0;
2072.
2073.         for (size_t j = 0; j < blen; j++) {
2074.
2075.             int pos = i + j;
2076.
2077.             int val = out[pos] + A[i] * B[j] + carry;
2078.
2079.             out[pos] = val % BASE_81;
2080.
2081.             carry = val / BASE_81;
2082.
2083.         }
2084.
2085.         out[i + blen] += carry;
2086.
2087.     }
2088.
2089. }
2090.
2091.
2092. static void add_shifted(unsigned char *dest, size_t dlen,
2093.
2094.                       const unsigned char *src, size_t slen,
2095.
2096.                       size_t shift) {
2097.
2098.     int carry = 0;
2099.
2100.     for (size_t i = 0; i < slen; i++) {
2101.
2102.         size_t idx = i + shift;
2103.
2104.         if (idx >= dlen) break;

```

```

2105.
2106.     int sum = dest[idx] + src[i] + carry;
2107.
2108.     dest[idx] = sum % BASE_81;
2109.
2110.     carry = sum / BASE_81;
2111.
2112. }
2113.
2114. size_t idx = slen + shift;
2115.
2116. while (carry && idx < dlen) {
2117.
2118.     int sum = dest[idx] + carry;
2119.
2120.     dest[idx] = sum % BASE_81;
2121.
2122.     carry = sum / BASE_81;
2123.
2124.     idx++;
2125.
2126. }
2127.
2128.}
2129.
2130.
2131. static void sub_inplace(unsigned char* out, const unsigned char* src, size_t length) {
2132.
2133.     int borrow = 0;
2134.
2135.     for (size_t i = 0; i < length; i++) {
2136.
2137.         int diff = out[i] - src[i] - borrow;
2138.
2139.         if (diff < 0) { diff += BASE_81; borrow = 1; } else { borrow = 0; }
2140.
2141.         out[i] = diff;
2142.
2143.     }
2144.
2145.}
2146.
2147.
2148. static void karatsuba(const unsigned char *A, const unsigned char *B, size_t n,
    unsigned char *out) {
2149.
2150.     if (n <= 16) { naive_mul(A, n, B, n, out); return; }
2151.
2152.     size_t half = n / 2, r = n - half;
2153.
2154.     const unsigned char *A0 = A, *A1 = A + half;
2155.
2156.     const unsigned char *B0 = B, *B1 = B + half;
2157.

```

```

2158. size_t len2 = 2 * n;
2159.
2160. unsigned char *p1 = calloc(len2, 1);
2161.
2162. unsigned char *p2 = calloc(len2, 1);
2163.
2164. unsigned char *p3 = calloc(len2, 1);
2165.
2166. unsigned char *sumA = calloc(n, 1);
2167.
2168. unsigned char *sumB = calloc(n, 1);
2169.
2170. karatsuba(A0, B0, half, p1);
2171.
2172. karatsuba(A1, B1, r, p2);
2173.
2174. memcpy(sumA, A1, r);
2175.
2176. for (size_t i = 0; i < half; i++) {
2177.     int s = sumA[i] + A0[i];
2178.     sumA[i] = s % BASE_81;
2179.     int c = s / BASE_81;
2180.     if (c && i + 1 < n) sumA[i + 1] += c;
2181. }
2182.
2183. memcpy(sumB, B1, r);
2184.
2185. for (size_t i = 0; i < half; i++) {
2186.     int s = sumB[i] + B0[i];
2187.     sumB[i] = s % BASE_81;
2188.     int c = s / BASE_81;
2189.     if (c && i + 1 < n) sumB[i + 1] += c;
2190. }
2191.
2192. karatsuba(sumA, sumB, n, p3);
2193.
2194. sub_inplace(p3, p1, len2);
2195.
2196. sub_inplace(p3, p2, len2);
2197.
2198. memset(out, 0, len2);
2199.
2200. add_shifted(out, len2, p1, len2, 0);
2201.
2202.

```

```

2212. add_shifted(out, len2, p3, len2, half);
2213.
2214. add_shifted(out, len2, p2, len2, 2 * half);
2215.
2216. free(p1); free(p2); free(p3);
2217.
2218. free(sumA); free(sumB);
2219.
2220.}
2221.
2222.
2223. static TernaryError t81bigint_karatsuba_multiply(const T81BigInt *a, const T81BigInt
    *b, T81BigInt *out) {
2224.
2225.     if ((a->len == 1 && a->digits[0] == 0) || (b->len == 1 && b->digits[0] == 0)) {
2226.
2227.         if (allocate_digits(out, 1) != TERNARY_NO_ERROR) return
            TERNARY_ERR_MEMALLOC;
2228.
2229.         out->digits[0] = 0; out->sign = 0;
2230.
2231.         return TERNARY_NO_ERROR;
2232.
2233.     }
2234.
2235.     size_t n = (a->len > b->len ? a->len : b->len);
2236.
2237.     unsigned char *A = calloc(n, 1), *B = calloc(n, 1);
2238.
2239.     if (!A || !B) { free(A); free(B); return TERNARY_ERR_MEMALLOC; }
2240.
2241.     memcpy(A, a->digits, a->len);
2242.
2243.     memcpy(B, b->digits, b->len);
2244.
2245.     size_t out_len = 2 * n;
2246.
2247.     unsigned char *prod = calloc(out_len, 1);
2248.
2249.     if (!prod) { free(A); free(B); return TERNARY_ERR_MEMALLOC; }
2250.
2251.     karatsuba(A, B, n, prod);
2252.
2253.     free(A); free(B);
2254.
2255.     out->sign = (a->sign != b->sign) ? 1 : 0;
2256.
2257.     while (out_len > 1 && prod[out_len - 1] == 0) out_len--;
2258.
2259.     if (allocate_digits(out, out_len) != TERNARY_NO_ERROR) { free(prod); return
        TERNARY_ERR_MEMALLOC; }
2260.
2261.     memcpy(out->digits, prod, out_len);
2262.

```

```

2263. free(prod);
2264.
2265. return TERNARY_NO_ERROR;
2266.
2267.}
2268.
2269.
2270. static int mul_cache_lookup(const char* key, T81BigInt *dst) {
2271.
2272.     for (int i = 0; i < MUL_CACHE_SIZE; i++) {
2273.
2274.         if (mul_cache[i].used && strcmp(mul_cache[i].key, key) == 0) {
2275.
2276.             if (allocate_digits(dst, mul_cache[i].result.len) != TERNARY_NO_ERROR) return
TERNARY_ERR_MEMALLOC;
2277.
2278.             dst->len = mul_cache[i].result.len;
2279.
2280.             dst->sign = mul_cache[i].result.sign;
2281.
2282.             memcpy(dst->digits, mul_cache[i].result.digits, dst->len);
2283.
2284.             return 0;
2285.
2286.         }
2287.
2288.     }
2289.
2290.     return 2;
2291.
2292.}
2293.
2294.
2295. static void mul_cache_store(const char* key, const T81BigInt *val) {
2296.
2297.     int slot = -1;
2298.
2299.     for (int i = 0; i < MUL_CACHE_SIZE; i++) {
2300.
2301.         if (!mul_cache[i].used) { slot = i; break; }
2302.
2303.     }
2304.
2305.     if (slot < 0) slot = 0;
2306.
2307.     strncpy(mul_cache[slot].key, key, sizeof(mul_cache[slot].key));
2308.
2309.     mul_cache[slot].key[sizeof(mul_cache[slot].key)-1] = '\0';
2310.
2311.     t81bigint_free(&mul_cache[slot].result);
2312.
2313.     mul_cache[slot].used = 1;
2314.
2315.     allocate_digits(&mul_cache[slot].result, val->len);

```

```

2316.
2317.     mul_cache[slot].result.len = val->len;
2318.
2319.     mul_cache[slot].result.sign = val->sign;
2320.
2321.     memcpy(mul_cache[slot].result.digits, val->digits, val->len);
2322.
2323. }
2324.
2325.
2326. static TernaryError multiply_with_cache(const T81BigInt *a, const T81BigInt *b,
    T81BigInt *out) {
2327.
2328.     char *as = NULL, *bs = NULL;
2329.
2330.     if (t81bigint_to_trit_string(a, &as) != TERNARY_NO_ERROR) return
        TERNARY_ERR_INVALID_INPUT;
2331.
2332.     if (t81bigint_to_trit_string(b, &bs) != TERNARY_NO_ERROR) { free(as); return
        TERNARY_ERR_INVALID_INPUT; }
2333.
2334.     char key[128];
2335.
2336.     snprintf(key, sizeof(key), "mul:%s:%s", as, bs);
2337.
2338.     free(as); free(bs);
2339.
2340.     if (mul_cache_lookup(key, out) == 0) return TERNARY_NO_ERROR;
2341.
2342.     TernaryError e = t81bigint_karatsuba_multiply(a, b, out);
2343.
2344.     if (e == TERNARY_NO_ERROR) { mul_cache_store(key, out); }
2345.
2346.     return e;
2347.
2348. }
2349.
2350.
2351. TernaryError tritjs_multiply_big(T81BigInt* a, T81BigInt* b, T81BigInt** result) {
2352.
2353.     if (!a || !b) return TERNARY_ERR_INVALID_INPUT;
2354.
2355.     *result = (T81BigInt*)calloc(1, sizeof(T81BigInt));
2356.
2357.     if (!*result) return TERNARY_ERR_MEMALLOC;
2358.
2359.     TernaryError e = multiply_with_cache(a, b, *result);
2360.
2361.     if (e != TERNARY_NO_ERROR) { free(*result); *result = NULL; }
2362.
2363.     return e;
2364.
2365. }
2366.

```

```

2367.
2368. TernaryError tritjs_factorial_big(T81BigInt* a, T81BigInt** result) {
2369.
2370.     if (!a) return TERNARY_ERR_INVALID_INPUT;
2371.
2372.     if (a->sign) return TERNARY_ERR_NEGATIVE;
2373.
2374.     if (a->len != 1 || a->digits[0] >= BASE_81) return TERNARY_ERR_OVERFLOW;
2375.
2376.     int val = a->digits[0];
2377.
2378.     if (val > 20) return TERNARY_ERR_OVERFLOW;
2379.
2380.     long long f = 1;
2381.
2382.     for (int i = 1; i <= val; i++) f *= i;
2383.
2384.     *result = (T81BigInt*)calloc(1, sizeof(T81BigInt));
2385.
2386.     if (!*result) return TERNARY_ERR_MEMALLOC;
2387.
2388.     if (allocate_digits(*result, 1) != TERNARY_NO_ERROR) { free(*result); return
TERNARY_ERR_MEMALLOC; }
2389.
2390.     (*result)->digits[0] = 0; (*result)->sign = 0;
2391.
2392.     while (f > 0) {
2393.
2394.         int digit = f % BASE_81;
2395.
2396.         f /= BASE_81;
2397.
2398.         int carry = digit;
2399.
2400.         size_t i = 0;
2401.
2402.         while (carry) {
2403.
2404.             int val2 = (*result)->digits[i] + carry;
2405.
2406.             (*result)->digits[i] = val2 % BASE_81;
2407.
2408.             carry = val2 / BASE_81;
2409.
2410.             i++;
2411.
2412.             if (i >= (*result)->len && carry) {
2413.
2414.                 if (allocate_digits(*result, (*result)->len + 1) != TERNARY_NO_ERROR) {
2415.
2416.                     tritbig_free(*result); *result = NULL; return TERNARY_ERR_MEMALLOC;
2417.
2418.                 }
2419.

```

```

2420.     }
2421.
2422.     }
2423.
2424. }
2425.
2426. while ((*result)->len > 1 && (*result)->digits[(*result)->len - 1] == 0)
2427.
2428.     (*result)->len--;
2429.
2430. return TERNARY_NO_ERROR;
2431.
2432.}
2433.
2434.
2435. TernaryError tritjs_power_big(T81BigInt* base, T81BigInt* exp, T81BigInt** result) {
2436.
2437.     if (!base || !exp) return TERNARY_ERR_INVALID_INPUT;
2438.
2439.     if (exp->sign) return TERNARY_ERR_NEGATIVE;
2440.
2441.     if (exp->len != 1 || exp->digits[0] >= BASE_81) return TERNARY_ERR_OVERFLOW;
2442.
2443.     int e = exp->digits[0];
2444.
2445.     if (e > 1000) return TERNARY_ERR_OVERFLOW;
2446.
2447.     *result = (T81BigInt*)calloc(1, sizeof(T81BigInt));
2448.
2449.     if (!*result) return TERNARY_ERR_MEMALLOC;
2450.
2451.     if (allocate_digits(*result, 1) != TERNARY_NO_ERROR) { free(*result); return
TERNARY_ERR_MEMALLOC; }
2452.
2453.     (*result)->digits[0] = 1; (*result)->sign = 0;
2454.
2455.     for (int i = 0; i < e; i++) {
2456.
2457.         T81BigInt *tmp = NULL;
2458.
2459.         TernaryError err = multiply_with_cache(*result, base, *result);
2460.
2461.         if (err != TERNARY_NO_ERROR) { tritbig_free(*result); free(*result); *result = NULL;
return err; }
2462.
2463.     }
2464.
2465.     if (base->sign && (e % 2) == 1)
2466.
2467.         (*result)->sign = 1;
2468.
2469.     return TERNARY_NO_ERROR;
2470.
2471.}

```

```

2472.
2473.
2474. /* Scientific functions via double conversion */
2475.
2476. static double t81bigint_to_double(T81BigInt* x) {
2477.
2478.     int sign = x->sign ? -1 : 1;
2479.
2480.     double accum = 0.0;
2481.
2482.     for (ssize_t i = x->len - 1; i >= 0; i--) {
2483.
2484.         accum = accum * BASE_81 + x->digits[i];
2485.
2486.     }
2487.
2488.     return sign * accum;
2489.
2490. }
2491.
2492.
2493. static T81BigInt* double_to_t81bigint(double d) {
2494.
2495.     T81BigInt* result = (T81BigInt*)calloc(1, sizeof(T81BigInt));
2496.
2497.     if (!result) return NULL;
2498.
2499.     int sign = (d < 0) ? 1 : 0;
2500.
2501.     if (d < 0) d = -d;
2502.
2503.     size_t capacity = 16;
2504.
2505.     result->digits = (unsigned char*)calloc(capacity, 1);
2506.
2507.     result->len = 0;
2508.
2509.     while (d >= 1.0) {
2510.
2511.         int digit = (int)fmod(d, BASE_81);
2512.
2513.         if (result->len >= capacity) { capacity *= 2; result->digits = realloc(result->digits,
2514. capacity); }
2515.
2516.         result->digits[result->len++] = (unsigned char) digit;
2517.
2518.         d = floor(d / BASE_81);
2519.     }
2520.
2521.     if (result->len == 0) { result->digits[0] = 0; result->len = 1; }
2522.
2523.     result->sign = sign;
2524.

```

```

2525.     return result;
2526.
2527.}
2528.
2529.
2530 TernaryError tritjs_sqrt_complex(T81BigInt* a, int precision, T81Complex* result) {
2531.
2532.     (void) precision;
2533.
2534.     double d = t81bigint_to_double(a);
2535.
2536.     double sq = sqrt(d);
2537.
2538.     T81BigInt* res = double_to_t81bigint(sq);
2539.
2540.     result->real = *res;
2541.
2542.     free(res);
2543.
2544.     result->imag.digits = calloc(1, 1);
2545.
2546.     result->imag.len = 1;
2547.
2548.     result->imag.sign = 0;
2549.
2550.     return TERNARY_NO_ERROR;
2551.
2552.}
2553.
2554.
2555 TernaryError tritjs_log3_complex(T81BigInt* a, int precision, T81Complex* result) {
2556.
2557.     (void) precision;
2558.
2559.     double d = t81bigint_to_double(a);
2560.
2561.     double l = log(d) / log(3);
2562.
2563.     T81BigInt* res = double_to_t81bigint(l);
2564.
2565.     result->real = *res;
2566.
2567.     free(res);
2568.
2569.     result->imag.digits = calloc(1, 1);
2570.
2571.     result->imag.len = 1;
2572.
2573.     result->imag.sign = 0;
2574.
2575.     return TERNARY_NO_ERROR;
2576.
2577.}
2578.

```

```

2579.
2580. TernaryError tritjs_sin_complex(T81BigInt* a, int precision, T81Complex* result) {
2581.
2582.     (void) precision;
2583.
2584.     double d = t81bigint_to_double(a);
2585.
2586.     double s = sin(d);
2587.
2588.     T81BigInt* res = double_to_t81bigint(s);
2589.
2590.     result->real = *res;
2591.
2592.     free(res);
2593.
2594.     result->imag.digits = calloc(1, 1);
2595.
2596.     result->imag.len = 1;
2597.
2598.     result->imag.sign = 0;
2599.
2600.     return TERNARY_NO_ERROR;
2601.
2602. }
2603.
2604.
2605. TernaryError tritjs_cos_complex(T81BigInt* a, int precision, T81Complex* result) {
2606.
2607.     (void) precision;
2608.
2609.     double d = t81bigint_to_double(a);
2610.
2611.     double c = cos(d);
2612.
2613.     T81BigInt* res = double_to_t81bigint(c);
2614.
2615.     result->real = *res;
2616.
2617.     free(res);
2618.
2619.     result->imag.digits = calloc(1, 1);
2620.
2621.     result->imag.len = 1;
2622.
2623.     result->imag.sign = 0;
2624.
2625.     return TERNARY_NO_ERROR;
2626.
2627. }
2628.
2629.
2630. TernaryError tritjs_tan_complex(T81BigInt* a, int precision, T81Complex* result) {
2631.
2632.     (void) precision;

```

```

2633.
2634.     double d = t81bigint_to_double(a);
2635.
2636.     double t = tan(d);
2637.
2638.     T81BigInt* res = double_to_t81bigint(t);
2639.
2640.     result->real = *res;
2641.
2642.     free(res);
2643.
2644.     result->imag.digits = calloc(1, 1);
2645.
2646.     result->imag.len = 1;
2647.
2648.     result->imag.sign = 0;
2649.
2650.     return TERNARY_NO_ERROR;
2651.
2652. }
2653.
2654.
2655. TernaryError tritjs_pi(int* len, int** pi) {
2656.
2657.     static int pi_val[] = {1, 0, 0, 1, 0, 2, 2, 1};
2658.
2659.     *len = 8;
2660.
2661.     *pi = malloc(8 * sizeof(int));
2662.
2663.     if (!*pi) return TERNARY_ERR_MEMALLOC;
2664.
2665.     memcpy(*pi, pi_val, 8 * sizeof(int));
2666.
2667.     return TERNARY_NO_ERROR;
2668.
2669. }
2670.
2671.
2672. /* Division using a simplified long division (via binary conversion) */
2673.
2674. TernaryError tritjs_divide_big(T81BigInt* a, T81BigInt* b, T81BigInt** quotient,
    T81BigInt** remainder) {
2675.
2676.     if (!a || !b) return TERNARY_ERR_INVALID_INPUT;
2677.
2678.     int b_zero = 1;
2679.
2680.     for (size_t i = 0; i < b->len; i++) {
2681.
2682.         if (b->digits[i] != 0) { b_zero = 0; break; }
2683.
2684.     }
2685.

```

```

2686.  if (b_zero) { log_error(TERNARY_ERR_DIVZERO, "tritjs_divide_big", __FILE__,
    __LINE__); return TERNARY_ERR_DIVZERO; }
2687.
2688.  *quotient = (T81BigInt*)calloc(1, sizeof(T81BigInt));
2689.
2690.  *remainder = (T81BigInt*)calloc(1, sizeof(T81BigInt));
2691.
2692.  if (!*quotient || !*remainder) return TERNARY_ERR_MEMALLOC;
2693.
2694.  if (allocate_digits(*remainder, a->len) != TERNARY_NO_ERROR) return
    TERNARY_ERR_MEMALLOC;
2695.
2696.  memcpy((*remainder)->digits, a->digits, a->len);
2697.
2698.  (*remainder)->len = a->len;
2699.
2700.  if (allocate_digits(*quotient, a->len) != TERNARY_NO_ERROR) return
    TERNARY_ERR_MEMALLOC;
2701.
2702.  memset((*quotient)->digits, 0, a->len);
2703.
2704.  int rem_int, b_int;
2705.
2706.  if (trit_to_binary(*remainder, &rem_int) != TERNARY_NO_ERROR) return
    TERNARY_ERR_INVALID_INPUT;
2707.
2708.  if (trit_to_binary(b, &b_int) != TERNARY_NO_ERROR) return
    TERNARY_ERR_INVALID_INPUT;
2709.
2710.  int q = rem_int / b_int;
2711.
2712.  int r = rem_int % b_int;
2713.
2714.  T81BigInt *q_big = NULL, *r_big = NULL;
2715.
2716.  if (binary_to_trit(q, &q_big) != TERNARY_NO_ERROR) return
    TERNARY_ERR_INVALID_INPUT;
2717.
2718.  if (binary_to_trit(r, &r_big) != TERNARY_NO_ERROR) return
    TERNARY_ERR_INVALID_INPUT;
2719.
2720.  *quotient = q_big;
2721.
2722.  *remainder = r_big;
2723.
2724.  return TERNARY_NO_ERROR;
2725.
2726.}
2727.
2728.
2729./* Main function: ncurses-based UI for command processing */
2730.
2731.int main(int argc, char** argv) {
2732.

```

```

2733. (void)argc; (void)argv;
2734.
2735. init_audit_log();
2736.
2737. initscr();
2738.
2739. cbreak();
2740.
2741. noecho();
2742.
2743. keypad(stdscr, TRUE);
2744.
2745.printw("TritJS-CISA-Optimized Version %s\n", VERSION);
2746.
2747.printw("Enter commands (type 'help' for assistance, 'exit' to quit):\n");
2748.
2749. refresh();
2750.
2751. char command[256];
2752.
2753. while (1) {
2754.
2755.     printw("> ");
2756.
2757.     getnstr(command, sizeof(command)-1);
2758.
2759.     if (strcmp(command, "exit") == 0) break;
2760.
2761.     if (strcmp(command, "help") == 0) {
2762.
2763.         printw("\nAvailable Commands:\n");
2764.
2765.         printw(" help  : Display this help message.\n");
2766.
2767.         printw(" exit  : Quit the utility.\n");
2768.
2769.         printw(" [other commands would be implemented here]\n\n");
2770.
2771.     } else {
2772.
2773.         printw("Command received: %s\n", command);
2774.
2775.         /* Command parsing and execution logic would be added here. */
2776.
2777.     }
2778.
2779.     refresh();
2780.
2781. }
2782.
2783. endwin();
2784.
2785. return 0;

```

```

2786.
2787.}
2788.
2789.
2790.* End of User-Space Utility section. */
2791.
2792.#endif /* !__KERNEL__ */
2793.
2794.
2795.*
=====
=====
2796.
2797. End of Ternary System Literate Program
2798.
2799. This file integrates the Axion kernel module and the TritJS-CISA-Optimized
2800.
2801. user-space utility into a single unified source file. It provides comprehensive
2802.
2803. documentation, a built-in help system, and shared ternary logic definitions to
2804.
2805. further our mission to transition from binary to native ternary computing.
2806.
2807.
=====
===== */
2808.
2809.

```

*Special thanks to xAI and OpenAI
for making this possible.*

